

MODULE 38

SQL and Query Performance

Master SQL fundamentals and advanced optimization techniques to improve query performance, reduce response time, and build efficient database solutions in enterprise environments.







MODULE 38 OVERVIEW

Master SQL fundamentals and advanced optimization techniques to improve query performance, reduce response time, and build efficient database solutions.

Every application depends on the database. This module covers writing correct SQL, combining tables with joins, using indexes and execution plans, and tuning real PostgreSQL queries -- backed by a hands-on evidence-based lab.

DURATION & LAB



1 Hour Theory

SQL review, all four JOIN types, indexes, execution plans, and query tuning strategies for PostgreSQL.



2 Hours Lab

PostgreSQL SQL and CRM Query Performance: real DBMS_XPLAN plans, justified indexes, sargable rewrites, and paging.



Scenario

As a Database Performance Engineer for an e-commerce platform, analyze slow queries and optimize database objects.

MODULE ARC



Master SQL & Joins

Review SELECT/WHERE/ORDER BY and combine tables with all four JOIN types.



Indexes & Plans

Understand clustered/non-clustered indexes and read EXPLAIN / EXPLAIN ANALYZE plans.



Tune & Monitor

Apply tuning strategies, avoid full table scans, and monitor SQL performance in production.

KEY TAKEAWAY Total time: 3 hours (1 hour theory + 2 hours hands-on lab). By the end, you will write efficient SQL, optimize performance, and keep PostgreSQL databases fast, reliable, and scalable.

WHY QUERY OPTIMIZATION MATTERS (1 OF 2)

Every application depends on the database. Inefficient SQL queries can slow down systems, increase costs, and impact user experience.

Optimized queries ensure your database performs at its best -- today and as your data grows. A single `SELECT * FROM orders WHERE status = 'Shipped'` can be fast or crippling depending on how it's written.

THE COST OF POOR QUERIES



Slow Response Times

Users experience delays and timeouts, leading to frustration and lost productivity.



Higher Infrastructure Costs

Inefficient queries consume more CPU, memory, and I/O, requiring more hardware.



Poor Scalability

As data grows, poorly written queries degrade performance and limit your ability to scale.



Business Impact

Slow reports, delayed decisions, and missed SLAs can directly affect business outcomes.



Harder to Maintain

Complex, unoptimized queries are difficult to maintain and increase the risk of errors.

KEY TAKEAWAY Slow queries don't just feel sluggish -- they chain directly into higher infrastructure costs, worse scalability, real business impact, and harder maintenance.

Why Query Optimization Matters

Efficient queries power faster applications, lower costs, and happier users.



Better Performance

Optimized queries run faster and reduce response times for users.



Lower Resource Usage

Uses less CPU, memory, and disk I/O, reducing load on the database server.



Reduced Costs

Efficient queries mean smaller infrastructure needs and lower cloud or hardware costs.



Scalability

A well-optimized database handles more users and transactions as your business grows.



Improved Reliability

Optimized queries reduce timeouts, blocking, and deadlocks, leading to more stable systems.

Without Query Optimization



Slow response times



High CPU and I/O usage



Poor user experience



Higher operational costs



Inefficient Query



With Query Optimization



Fast response times



Low CPU and I/O usage



Better user experience



Lower operational costs



Optimized Query

WHY QUERY OPTIMIZATION MATTERS (2 OF 2)

The benefits of query optimization are the direct mirror image of the costs on the previous slide -- faster, cheaper, more scalable, more reliable.

THE BENEFITS OF QUERY OPTIMIZATION



Faster Performance

Return results quickly and improve overall application speed.



Lower Costs

Use resources efficiently and reduce infrastructure and operational costs.



Better Scalability

Handle more data and users without sacrificing performance.



Reliability & Stability

Consistent performance helps meet SLAs and improves system reliability.



Better User Experience

Happy users, faster workflows, and stronger business impact.

KEY TAKEAWAY Optimizing your SQL queries is not just a technical task -- it's a critical factor in delivering fast, reliable, and cost-effective database solutions that drive business success.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to write efficient SQL, optimize performance, and ensure your PostgreSQL databases are fast, reliable, and scalable.

- **1. Understand SQL Fundamentals** — review core SQL concepts including SELECT statements, filtering, sorting, and basic query structure.
- **2. Work with Joins** — use INNER, LEFT, RIGHT, and FULL OUTER JOINS to combine data from multiple tables effectively.
- **3. Understand Indexes** — explain the role of indexes and differentiate between clustered and non-clustered indexes.
- **4. Analyze Query Execution** — read and interpret query execution plans to understand how queries are processed.
- **5. Identify Performance Issues** — identify slow queries and understand common causes such as missing indexes and full table scans.
- **6. Apply Optimization Techniques** — use query tuning strategies to improve query performance and reduce resource consumption.
- **7. Avoid Full Table Scans** — design queries and indexes that help the database engine retrieve data efficiently.
- **8. Design Efficient Queries** — write clear, efficient, and maintainable SQL that delivers accurate results with optimal performance.
- **9. Follow Database Best Practices** — apply enterprise-level best practices for SQL design, indexing, and performance tuning in production.
- **10. Monitor SQL Performance** — use tools and techniques to monitor, measure, and maintain database performance in production.

KEY TAKEAWAY Module Outcome: you will be able to write efficient SQL queries, optimize performance, and ensure your PostgreSQL databases are fast, reliable, and scalable.

SQL REVIEW

SQL (Structured Query Language) is the standard language used to communicate with relational databases -- store, retrieve, update, and manage data efficiently.

SQL CATEGORIES

Category	Examples
DDL — Data Definition	CREATE, ALTER, DROP, TRUNCATE
DML — Data Manipulation	INSERT, UPDATE, DELETE
DQL — Data Query	SELECT
DCL — Data Control	GRANT, REVOKE
TCL — Transaction Control	COMMIT, ROLLBACK, SAVEPOINT

BASIC SQL SYNTAX

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition  
GROUP BY column_name  
HAVING condition  
ORDER BY column_name ASC|DESC  
LIMIT number;
```

EXAMPLE: SIMPLE SELECT QUERY

```
SELECT customer_id, first_name, last_name, email  
FROM customers  
WHERE status = 'Active'  
ORDER BY last_name ASC  
LIMIT 10;
```

SELECT

FROM

WHERE

JOIN

GROUP BY

HAVING

ORDER BY

DISTINCT

LIMIT

KEY TAKEAWAY SQL is the foundation of working with relational databases. In this module, we build on these basics and focus on writing high-performance PostgreSQL SQL.

SQL Review

Structured Query Language – The standard language for managing and querying relational databases.



1. Data Definition (DDL)

CREATE TABLE

```
CREATE TABLE Customers (  
  CustomerID INT PRIMARY KEY,  
  FirstName VARCHAR(50) NOT NULL,  
  LastName VARCHAR(50) NOT NULL,  
  Email VARCHAR(100) UNIQUE,  
  CreatedAt DATETIME DEFAULT GETDATE()  
);
```

ALTER TABLE

```
ALTER TABLE Customers  
  ADD Phone VARCHAR(20);
```

DROP TABLE

```
DROP TABLE Customers;
```



2. Data Manipulation (DML)

INSERT

```
INSERT INTO Customers  
  (FirstName, LastName, Email)  
VALUES ('John', 'Doe', 'john@example.com');
```

UPDATE

```
UPDATE Customers  
SET Email = 'john.doe@example.com'  
WHERE CustomerID = 1;
```

DELETE

```
DELETE FROM Customers  
WHERE CustomerID = 1;
```



3. Querying Data (SELECT)

SELECT All Columns

```
SELECT * FROM Customers;
```

SELECT Specific Columns

```
SELECT FirstName, LastName, Email  
FROM Customers;
```

WHERE Clause

```
SELECT * FROM Customers  
WHERE Country = 'USA';
```

ORDER BY

```
SELECT * FROM Customers  
ORDER BY LastName ASC;
```

DISTINCT

```
SELECT DISTINCT Country  
FROM Customers;
```

LIMIT / TOP

```
SELECT TOP 10 *  
FROM Customers;
```



4. Filtering Data

WHERE with Operators

```
SELECT * FROM Customers  
WHERE Age > 30 AND Country = 'USA';
```

IN Operator

```
SELECT * FROM Customers  
WHERE Country IN ('USA', 'Canada', 'UK');
```

LIKE Operator

```
SELECT * FROM Customers  
WHERE LastName LIKE 'S%';
```

BETWEEN Operator

```
SELECT * FROM Orders  
WHERE OrderDate BETWEEN '2024-01-01' AND '2024-12-31';
```



5. Aggregate Functions

COUNT

```
SELECT COUNT(*) AS TotalCustomers FROM Customers;
```

SUM

```
SELECT SUM(TotalAmount) AS TotalSales FROM Orders;
```

AVG

```
SELECT AVG(Age) AS AverageAge FROM Customers;
```

MIN / MAX

```
SELECT MIN(OrderDate) AS FirstOrder,  
       MAX(OrderDate) AS LastOrder  
FROM Orders;
```



6. Grouping Data

GROUP BY

```
SELECT Country, COUNT(*) AS TotalCustomers  
FROM Customers  
GROUP BY Country;
```

HAVING Clause

```
SELECT Country, COUNT(*) AS TotalCustomers  
FROM Customers  
GROUP BY Country  
HAVING COUNT(*) > 5;
```



7. Joins

INNER JOIN

```
SELECT o.OrderID, c.FirstName, c.LastName  
FROM Orders o  
INNER JOIN Customers c  
ON o.CustomerID = c.CustomerID;
```

LEFT JOIN

```
SELECT c.FirstName, o.OrderID  
FROM Customers c  
LEFT JOIN Orders o  
ON c.CustomerID = o.CustomerID;
```



8. Subqueries & Set Operations

Subquery in WHERE

```
SELECT * FROM Customers  
WHERE CustomerID IN (  
  SELECT CustomerID FROM Orders  
  WHERE TotalAmount > 1000  
);
```

EXISTS

```
SELECT * FROM Customers c  
WHERE EXISTS (  
  SELECT 1 FROM Orders o  
  WHERE o.CustomerID = c.CustomerID  
);
```

UNION

```
SELECT Email FROM Customers_US  
UNION  
SELECT Email FROM Customers_UK;
```

INTERSECT

```
SELECT Email FROM Customers_A  
INTERSECT  
SELECT Email FROM Customers_B;
```

EXCEPT

```
SELECT Email FROM Customers_A  
EXCEPT  
SELECT Email FROM Customers_B;
```



9. Useful Functions

String Functions

UPPER(text), LOWER(text),
LEN(text), SUBSTRING(text, start, len)

Date Functions

GETDATE(), DATEADD(), DATEDIFF(),
CONVERT(), YEAR(), MONTH(), DAY()

Numeric Functions

ROUND(number, decimals),
ABS(number), CEILING(), FLOOR()

Null Handling

ISNULL(value, replacement),
COALESCE(value1, value2, ...)



Best Practices



Use meaningful table and column names



Use indexes on frequently queried columns



Avoid SELECT * in production



Use parameters to prevent SQL injection



Test and optimize queries regularly

SELECT STATEMENTS

The SELECT statement retrieves data from one or more tables. It is the most frequently used SQL statement.

BASIC SYNTAX

```
SELECT column1, column2, ...  
FROM table_name;
```

- **SELECT** — specifies the columns to retrieve.
- **column1, column2, ...** — one or more column names, or * for all columns.
- **FROM** — specifies the table (or view) to retrieve data from.

COMMON VARIATIONS

Variation	Example
Remove duplicates	SELECT DISTINCT status FROM orders;
Limit rows	SELECT * FROM orders LIMIT 10;
Filter + sort	SELECT * FROM orders WHERE status = 'Shipped' ORDER BY order_date DESC;

EXAMPLES

Description	SQL Query
Select all columns	SELECT * FROM customers;
Select specific columns	SELECT first_name, email FROM customers;
Select with column alias	SELECT first_name AS name, email AS contact FROM customers;

KEY TAKEAWAYS

- SELECT is used to retrieve data from the database.
- You can select all columns (*) or specific columns.
- Use aliases (AS) to make result sets easier to read.
- Combine with WHERE, ORDER BY, and LIMIT to get the exact data you need.

KEY TAKEAWAY Best Practice: always select only the columns you need instead of using * to improve performance and reduce network traffic.



SELECT Statements

Retrieve data from one or more tables in a database.

Basic Syntax



```
SELECT column1, column2, ...  
FROM table_name  
[WHERE condition]  
[ORDER BY column_name [ASC | DESC]]  
[LIMIT number];
```

- **SELECT** specifies columns to retrieve.
- **FROM** specifies the table.
- Optional clauses filter, sort, and limit results.

Common Clauses



WHERE
Filter rows that meet a condition.



ORDER BY
Sort the result set.
ASC (ascending) or DESC (descending)

123

LIMIT / TOP
Limit the number of rows returned.



DISTINCT
Return only unique values.

Examples

1 Select All Columns

```
SELECT *  
FROM Customers;
```

CustomerID	FirstName	LastName	Email
1	John	Doe	john@example.com
2	Jane	Smith	jane@example.com
3	Bob	Brown	bob@example.com

2 Select Specific Columns

```
SELECT FirstName, LastName, Email  
FROM Customers;
```

FirstName	LastName	Email
John	Doe	john@example.com
Jane	Smith	jane@example.com
Bob	Brown	bob@example.com

3 With WHERE Clause

```
SELECT *  
FROM Customers  
WHERE Country = 'USA';
```

CustomerID	FirstName	LastName	Country
1	John	Doe	USA
3	Bob	Brown	USA

4 With ORDER BY Clause

```
SELECT FirstName, LastName  
FROM Customers  
ORDER BY LastName ASC;
```

FirstName	LastName
Bob	Brown
John	Doe
Jane	Smith

5 Using DISTINCT

```
SELECT DISTINCT Country  
FROM Customers;
```

Country
USA
Canada
UK
Australia

6 With LIMIT (Top N Rows)

```
SELECT FirstName, LastName  
FROM Customers  
ORDER BY CustomerID  
LIMIT 2;
```

FirstName	LastName
John	Doe
Jane	Smith

Best Practices



Select only the columns you need.



Use **WHERE** to filter early.



Index columns used in **WHERE** and **ORDER BY**.



Avoid **SELECT *** in production.



Test and optimize queries regularly.

FILTERING DATA WITH WHERE

The *WHERE* clause filters rows based on a specified condition. Only rows where the condition evaluates to *TRUE* are returned.

COMMON COMPARISON OPERATORS

Operator	Example
=	WHERE age = 30
<> or !=	WHERE status <> 'Closed'
> / <	WHERE amount > 1000
>= / <=	WHERE salary >= 50000
BETWEEN	WHERE age BETWEEN 18 AND 30
IN	WHERE status IN ('New','Open')
LIKE	WHERE name LIKE 'A%'
IS NULL / IS NOT NULL	WHERE email IS NULL

EXAMPLES

Description	SQL Query
Filter by equality	WHERE city = 'Toronto'
Filter by range	WHERE order_date >= '2024-01-01' AND order_date <= '2024-03-31'
Multiple conditions	WHERE category = 'Electronics' AND price > 500
Filter using IN	WHERE status IN ('Active','Pending')
Filter NULL values	WHERE last_login IS NULL

KEY TAKEAWAYS

- WHERE filters rows based on conditions.
- Combine operators and logical keywords (AND, OR, NOT) for complex filters.
- Filtering data early improves performance by reducing the result set size.

KEY TAKEAWAY Best Practice: use indexes on columns used in the WHERE clause to improve query performance.

Filtering Data with WHERE

Use the WHERE clause to filter rows based on specific conditions.

Overview

The **WHERE** clause filters rows and returns only those that satisfy the specified condition(s).



```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

Comparison Operators

=	Equal to
<> or !=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
BETWEEN	Between a range
IN	Match any value in a list
LIKE	Pattern matching
IS NULL	IS NULL
IS NOT NULL	Is not NULL

Best Practices



Filter data as early as possible to improve performance.



Use indexes on columns used in WHERE clauses.



Avoid functions on columns in WHERE (e.g., WHERE YEAR(DateCol) = 2024).



Test and optimize queries regularly.



Use appropriate data types for accurate filtering.

Examples

1 Equal To

```
SELECT *  
FROM Employees  
WHERE Department = 'Sales';
```

EmpID	EmpName	Department	Salary
101	Alice	Sales	60000
104	David	Sales	55000

2 Greater Than

```
SELECT *  
FROM Employees  
WHERE Salary > 60000;
```

EmpID	EmpName	Department	Salary
102	Bob	IT	75000
105	Eva	HR	65000

3 Between a Range

```
SELECT *  
FROM Employees  
WHERE Salary BETWEEN 50000 AND 70000;
```

EmpID	EmpName	Department	Salary
101	Alice	Sales	60000
103	Charlie	IT	50000
104	David	Sales	55000
105	Eva	HR	65000

4 IN Operator

```
SELECT *  
FROM Employees  
WHERE Department IN ('Sales', 'HR');
```

EmpID	EmpName	Department	Salary
101	Alice	Sales	60000
104	David	Sales	55000
105	Eva	HR	65000

5 LIKE Operator

```
SELECT *  
FROM Employees  
WHERE EmpName LIKE 'A%';
```

EmpID	EmpName	Department	Salary
101	Alice	Sales	60000

'A%' matches names that start with 'A'.

6 IS NULL

```
SELECT *  
FROM Employees  
WHERE ManagerID IS NULL;
```

EmpID	EmpName	Department	ManagerID
101	Alice	Sales	NULL

Returns rows where ManagerID is NULL.

7 Combined Conditions

```
SELECT *  
FROM Employees  
WHERE Department = 'IT'  
AND Salary >= 60000;
```

EmpID	EmpName	Department	Salary
102	Bob	IT	75000

Use AND, OR, and parentheses to combine conditions.

8 Not Equal To

```
SELECT *  
FROM Employees  
WHERE Department <> 'Sales';
```

EmpID	EmpName	Department	Salary
102	Bob	IT	75000
103	Charlie	IT	50000
105	Eva	HR	65000

SORTING DATA WITH ORDER BY

The *ORDER BY* clause sorts the result set in ascending (*ASC*) or descending (*DESC*) order based on one or more columns.

BASIC SYNTAX

```
SELECT column1, column2, ...  
FROM table_name  
ORDER BY column1 [ASC|DESC], column2 [ASC|DESC];
```

- **ASC** — ascending order is the default.
- **DESC** — descending order, most-recent/highest first.
- **Multiple columns** — sorted left to right in priority order.
- **NULL handling** — whether NULLs sort first or last is database dependent.

BEST PRACTICE

Always order by indexed columns when possible, especially on large datasets, to avoid an expensive in-memory sort.

EXAMPLES

Description	SQL Query
Sort one column (ASC)	ORDER BY first_name ASC
Sort one column (DESC)	ORDER BY first_name DESC
Sort multiple columns	ORDER BY city ASC, order_date ASC
Mixed ASC and DESC	ORDER BY city ASC, order_date DESC

KEY TAKEAWAYS

- ORDER BY is used to sort the result set.
- ASC sorts ascending (default), DESC sorts descending.
- You can sort by multiple columns to achieve more specific ordering.
- Sorting large result sets can be resource intensive — use indexes and filters.

KEY TAKEAWAY ORDER BY sorts the result set by one or more columns — sort by indexed columns when possible on large datasets to avoid an expensive in-memory sort.

Sorting Data with ORDER BY

Use ORDER BY to sort the result set in ascending or descending order.

Overview

ORDER BY sorts the rows returned by a SELECT statement based on one or more columns.

```
SELECT column1, column2, ...  
FROM table_name  
ORDER BY column_name [ASC | DESC];
```

Sort Order



ASC (Ascending)

Default order (A → Z, 0 → 9)
NULLs appear first (varies by DBMS)



DESC (Descending)

Reverse order (Z → A, 9 → 0)
NULLs appear last (varies by DBMS)

Notes

- Default sort order is ASC if not specified.
- You can sort by column name or column position.
- ORDER BY is the last clause in the SELECT statement (before LIMIT/OFFSET if used).

Best Practices



Use indexes on columns used in ORDER BY.



Sort only the necessary data (avoid SELECT *).



ORDER BY can impact performance on large datasets.



Test and optimize queries regularly.



Be explicit with ASC/DESC for better readability.

Examples

1 Ascending Order (Default)

```
SELECT EmpID, EmpName, Salary  
FROM Employees  
ORDER BY EmpName;
```

EmpID	EmpName	Salary
101	Alice	60000
103	Charlie	50000
102	Bob	75000
104	David	55000

2 Descending Order

```
SELECT EmpID, EmpName, Salary  
FROM Employees  
ORDER BY Salary DESC;
```

EmpID	EmpName	Salary
102	Bob	75000
101	Alice	60000
104	David	55000
103	Charlie	50000

3 Sort by Multiple Columns

```
SELECT EmpID, EmpName, Department, Salary  
FROM Employees  
ORDER BY Department ASC, Salary DESC;
```

EmpID	EmpName	Department	Salary
101	Alice	Sales	60000
104	David	Sales	55000
103	Charlie	IT	50000
102	Bob	IT	75000

4 Sort by Column Position

```
SELECT EmpID, EmpName, Salary  
FROM Employees  
ORDER BY 3 DESC;
```

EmpID	EmpName	Salary
102	Bob	75000
101	Alice	60000
104	David	55000
103	Charlie	50000

5 Ascending and Descending Mix

```
SELECT EmpID, EmpName, Department, Salary  
FROM Employees  
ORDER BY Department ASC, Salary DESC;
```

EmpID	EmpName	Department	Salary
101	Alice	Sales	60000
104	David	Sales	55000
102	Bob	IT	75000
103	Charlie	IT	50000

6 NULL Handling (Example)

```
SELECT EmpID, EmpName, ManagerID  
FROM Employees  
ORDER BY ManagerID ASC;
```

EmpID	EmpName	ManagerID
103	Charlie	NULL
101	Alice	1
102	Bob	1
104	David	2

INNER JOIN

An INNER JOIN returns rows that have matching values in both tables based on the join condition. Non-matching rows are excluded.

SYNTAX

```
SELECT a.column1, b.column2, ...
FROM table_a a
INNER JOIN table_b b
    ON a.common_column = b.common_column;
```

KEY POINTS

- Returns only matching rows from both tables.
- Eliminates rows with NULLs in the join column from either table.
- The most commonly used join type.
- Efficient when proper indexes exist on the join columns.

USE CASES

Order + customer details

Valid relationships only

EXAMPLE — CUSTOMERS INNER JOIN ORDERS

customer_id	first_name	order_id	amount
101	Aman	2001	125.00
101	Aman	2003	320.50
103	Priya	2002	250.75

EXPLANATION

- Matches customers with their orders on customer_id.
- Customers 102 and 104 have no orders, so they are excluded.
- Order 2004 references customer_id 105, which does not exist, so it is excluded too.

KEY TAKEAWAY Best Practice: ensure indexes exist on join columns (e.g., customer_id) in both tables to keep INNER JOIN performance efficient.

INNER JOIN

Returns only the rows that have matching values in both tables.

Explanation

- INNER JOIN returns rows when there is at least one match in both tables based on the join condition.

Syntax

```
SELECT columns
FROM table1 t1
INNER JOIN table2 t2
ON t1.common_column = t2.common_column;
```

Example Tables

Employees

EmpID	EmpName	DeptID
1	Alice	10
2	Bob	20
3	Charlie	10
4	David	30
5	Eve	40

Departments

DeptID	DeptName	Location
10	HR	New York
20	Finance	Chicago
30	IT	Dallas
50	Marketing	Boston

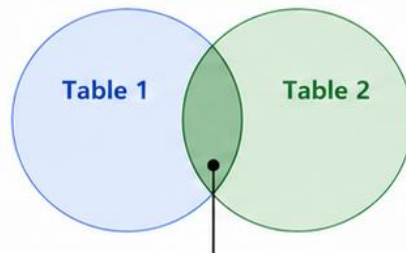
Result (INNER JOIN)

EmpID	EmpName	DeptName	Location
1	Alice	HR	New York
2	Bob	Finance	Chicago
3	Charlie	HR	New York
4	David	IT	Dallas



Employee with DeptID = 40 (Eve) is not included because there is no matching DeptID in Departments.

Visual Representation



INNER JOIN
Only matching rows
in both tables

SQL Example

```
SELECT e.EmpID, e.EmpName, d.DeptName, d.Location
FROM Employees e
INNER JOIN Departments d
ON e.DeptID = d.DeptID;
```



Key Points

- INNER JOIN returns only matching rows.
- Non-matching rows from either table are excluded.
- The join condition is specified in the ON clause.

LEFT JOIN

A *LEFT JOIN* returns all rows from the left table and the matching rows from the right table. Unmatched right-table columns return *NULL*.

SYNTAX

```
SELECT a.column1, b.column2, ...
FROM table_a a
LEFT JOIN table_b b
  ON a.common_column = b.common_column;
```

KEY POINTS

- Preserves all rows from the left table (table_a).
- Fills unmatched right-table columns with NULL.
- Useful for finding related data or identifying missing matches.
- Not always symmetrical — the order of tables matters.

USE CASES

All customers + orders (if any)

Customers with no orders

EXAMPLE — CUSTOMERS LEFT JOIN ORDERS

customer_id	first_name	order_id	amount
101	Aman	2001	125.00
101	Aman	2003	320.50
102	Neha	NULL	NULL
104	John	NULL	NULL

EXPLANATION

- Customer 101 has two orders → two rows returned.
- Customers 102 and 104 have no orders → order columns show NULL, but the customer row is kept.

KEY TAKEAWAY Best Practice: ensure an index exists on the right table's join column (e.g., orders.customer_id) to keep LEFT JOIN performance efficient.

LEFT JOIN

Returns all rows from the left table and matching rows from the right table.
If no match is found, the result contains NULLs for the right table columns.

EXPLANATION



LEFT JOIN returns all rows from the left table (table1) and the matched rows from the right table (table2). If no match exists, NULL values are returned for the right table columns.

EXAMPLE TABLES

Employees (Left Table)

EmpID	EmpName	DeptID
1	Alice	10
2	Bob	20
3	Charlie	10
4	David	30
5	Eve	40

Departments (Right Table)

DeptID	DeptName	Location
10	HR	New York
20	Finance	Chicago
30	IT	Dallas

RESULT (LEFT JOIN)

EmpID	EmpName	DeptName	Location
1	Alice	HR	New York
2	Bob	Finance	Chicago
3	Charlie	HR	New York
4	David	IT	Dallas
5	Eve	NULL	NULL



All rows from Employees are returned. Eve has no matching DeptID, so NULLs are shown for DeptName and Location.

SYNTAX

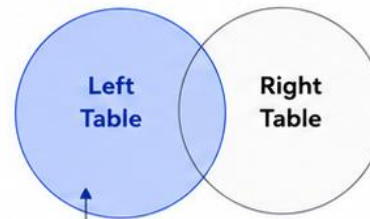
```
SELECT columns
FROM table1 t1
LEFT JOIN table2 t2
ON t1.common_column = t2.common_column;
```



KEY POINTS

- All rows from the left table are returned.
- Matching rows from the right table are included.
- Non-matching rows from the right table return NULLs.
- The join condition is specified in the ON clause.

VISUAL REPRESENTATION



All rows from
Left Table
(plus matching rows
from Right Table)

SQL EXAMPLE

```
SELECT e.EmpID, e.EmpName, d.DeptName, d.Location
FROM Employees e
LEFT JOIN Departments d
ON e.DeptID = d.DeptID
ORDER BY e.EmpID;
```



USE CASES

- List all employees with their department details.
- Find records in the left table even if there is no match in the right table.

RIGHT JOIN

A *RIGHT JOIN* returns all rows from the right table and the matching rows from the left table. Unmatched left-table columns return *NULL*.

SYNTAX

```
SELECT a.column1, b.column2, ...
FROM table_a a
RIGHT JOIN table_b b
  ON a.common_column = b.common_column;
```

KEY POINTS

- Preserves all rows from the right table (table_b).
- Fills unmatched left-table columns with NULL.
- Useful when the right table is the primary focus of the query.
- Not always symmetrical — the order of tables matters.

USE CASES

All orders, even unmatched

Right table as primary dataset

EXAMPLE — CUSTOMERS RIGHT JOIN ORDERS

customer_id	first_name	order_id	amount
101	Aman	2001	125.00
103	Priya	2002	250.75
101	Aman	2003	320.50
NULL	NULL	2004	150.25

EXPLANATION

- Order 2004 belongs to customer_id 105, which does not exist in customers.
- Since it has no match in the left table, the customer columns are NULL — but the order row is still kept because it exists in the right table.

KEY TAKEAWAY Best Practice: ensure indexes exist on join columns in both tables (customers.customer_id, orders.customer_id) to keep RIGHT JOIN performance efficient.

RIGHT JOIN

Returns all rows from the right table and matching rows from the left table.
If no match is found, the result contains NULLs for the left table columns.

EXPLANATION



RIGHT JOIN returns all rows from the right table (table2) and the matched rows from the left table (table1). If no match exists, NULL values are returned for the left table columns.

EXAMPLE TABLES

Employees (Left Table)

EmpID	EmpName	DeptID
1	Alice	10
2	Bob	20
3	Charlie	10
4	David	30

Departments (Right Table)

DeptID	DeptName	Location
10	HR	New York
20	Finance	Chicago
30	IT	Dallas
40	Marketing	Boston

RESULT (RIGHT JOIN)

EmpID	EmpName	DeptID	DeptName	Location
1	Alice	10	HR	New York
3	Charlie	10	HR	New York
2	Bob	20	Finance	Chicago
4	David	30	IT	Dallas
NULL	NULL	40	Marketing	Boston



Department with DeptID = 40 (Marketing) has no matching employees, so EmpID and EmpName are NULL.

SYNTAX

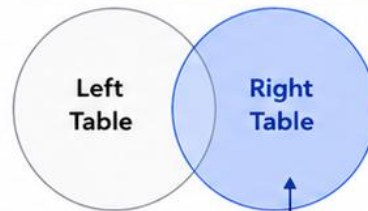
```
SELECT columns
FROM table1 t1
RIGHT JOIN table2 t2
ON t1.common_column = t2.common_column;
```



KEY POINTS

- All rows from the right table are returned.
- Matching rows from the left table are included.
- Non-matching rows from the left table return NULLs.
- The join condition is specified in the ON clause.

VISUAL REPRESENTATION



All rows from
Right Table
(plus matching rows
from Left Table)

SQL EXAMPLE

```
SELECT e.EmpID, e.EmpName, d.DeptName, d.Location
FROM Employees e
RIGHT JOIN Departments d
ON e.DeptID = d.DeptID
ORDER BY d.DeptID;
```



USE CASES

- List all departments and their employees.
- Find departments that have no employees.
- Reporting where the right table is the primary focus of analysis.

FULL OUTER JOIN

A *FULL OUTER JOIN* returns all rows from both tables — matching rows, plus non-matching rows from each side with *NULLs* filling the gaps.

SYNTAX

```
SELECT a.column1, b.column2, ...
FROM table_a a
FULL OUTER JOIN table_b b
  ON a.common_column = b.common_column;
```

KEY POINTS

- Returns all rows from both tables.
- Includes matching rows AND non-matching rows from both sides.
- Non-matching columns are filled with NULL.
- Useful for data comparison, reconciliation, and completeness checks.

BEST PRACTICE

- Use COALESCE() to handle NULLs in display or sorting.
- Be cautious with FULL OUTER JOIN on large tables — it can be expensive.
- Ensure proper indexing on join columns to improve performance.

EXAMPLE — CUSTOMERS FULL OUTER JOIN ORDERS

customer_id	first_name	order_id	amount
101	Aman	2001	125.00
102	Neha	NULL	NULL
103	Priya	2002	250.75
NULL	NULL	2003	320.50
NULL	NULL	2004	150.25

LEGEND

Matched (both)

Left only

Right only

KEY TAKEAWAY FULL OUTER JOIN = INNER JOIN matches + LEFT-only rows + RIGHT-only rows, all in one result set — reserved for reconciliation and completeness checks.

FULL OUTER JOIN

Returns all rows when there is a match in either left or right table.
If there is no match, the result contains NULLs for the missing side.

OVERVIEW

FULL OUTER JOIN combines the results of both LEFT JOIN and RIGHT JOIN.

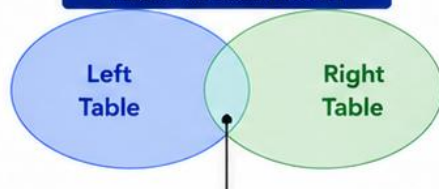
It returns:

- Matching rows from both tables
- Non-matching rows from the left table
- Non-matching rows from the right table

SYNTAX

```
SELECT columns
FROM table1 t1
FULL OUTER JOIN table2 t2
ON t1.common_column = t2.common_column;
```

VISUAL REPRESENTATION



FULL OUTER JOIN
(All rows from Left Table
+ Matching rows
+ All rows from Right Table)

EXAMPLE TABLES

Employees (Left Table)

EmpID	EmpName	DeptID
1	Alice	10
2	Bob	20
3	Charlie	10
4	David	30
5	Eve	40

Departments (Right Table)

DeptID	DeptName	Location
10	HR	New York
20	Finance	Chicago
30	IT	Dallas
40	Marketing	Boston
50	Legal	San Francisco

RESULT (FULL OUTER JOIN)

EmpID	EmpName	DeptID	DeptName	Location
1	Alice	10	HR	New York
2	Bob	20	Finance	Chicago
3	Charlie	10	HR	New York
4	David	30	IT	Dallas
5	Eve	40	Marketing	Boston
NULL	NULL	50	Legal	San Francisco



All rows from both tables are returned.
NULLs appear where there is no match.

SQL EXAMPLE

```
SELECT e.EmpID, e.EmpName, e.DeptID,
       d.DeptName, d.Location
FROM Employees e
FULL OUTER JOIN Departments d
ON e.DeptID = d.DeptID
ORDER BY COALESCE(e.EmpID, -1),
         COALESCE(d.DeptID, -1);
```

KEY POINTS



Returns all rows from both tables.



Matching rows are combined.



Non-matching rows from the left table
have NULLs for right table columns.



Non-matching rows from the right table
have NULLs for left table columns.



Equivalent to: (LEFT JOIN) UNION (RIGHT JOIN)
excluding duplicate matches.

TRY IT YOURSELF — EXERCISE 1: ACCESS PATTERNS

Reinforces: naming how the CRM will actually query customers and accounts before we learn indexes to support those queries.

STEPS (IN notes/lab38-perf.md)

Step	What To Do
1 — List Patterns	Write at least three real CRM access patterns: lookup by customer_id (CUS-1001), filter by status, filter by created_at range, and accounts by customer_id.
2 — Name the Hot Path	Mark the single hottest path: single-customer lookup by customer_id — the query every CRM screen calls first.
3 — Flag the Anti-Pattern	Write one line warning against SELECT * without a WHERE clause on a huge table in application code.
4 — Save Your Notes	Save the pattern list plus the named hot path to notes/lab38-perf.md.

Hot path vs. anti-pattern

```
-- Hot path: single-customer lookup
SELECT customer_id, full_name, status FROM customer WHERE customer_id = 'CUS-1001';
-- Anti-pattern: avoid in app code on a huge table
SELECT * FROM customer;
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-access-patterns.md (workspace: examples/module-38-exercises).

UNDERSTANDING INDEXES

Indexes are special data structures that improve the speed of data retrieval operations on a database table.

An index works like the index in a book — it helps the database find data quickly without scanning the entire table.

WHAT IS AN INDEX?

- A data structure (e.g., B-Tree, Hash) that stores a small portion of the table data in a sorted way.
- Contains a copy of the indexed column value and a pointer to the actual row.
- The database uses it to locate rows quickly without scanning the whole table.

WHY INDEXES IMPROVE PERFORMANCE

- **Reduces full table scans** — database locates rows via the index instead of reading every row.
- **Speeds up filters and sorts** — helps WHERE, JOIN, ORDER BY, GROUP BY, DISTINCT.
- **Efficient data access** — less disk I/O and CPU usage.

TYPES OF INDEXES

Type	Used For
B-Tree	Most common; equality and range queries (>, <, BETWEEN, ORDER BY).
Hash	Equality (=) lookups only — not used for range queries.
Composite	Created on multiple columns, e.g. (last_name, first_name).
Unique	Ensures all values in the indexed column are unique.

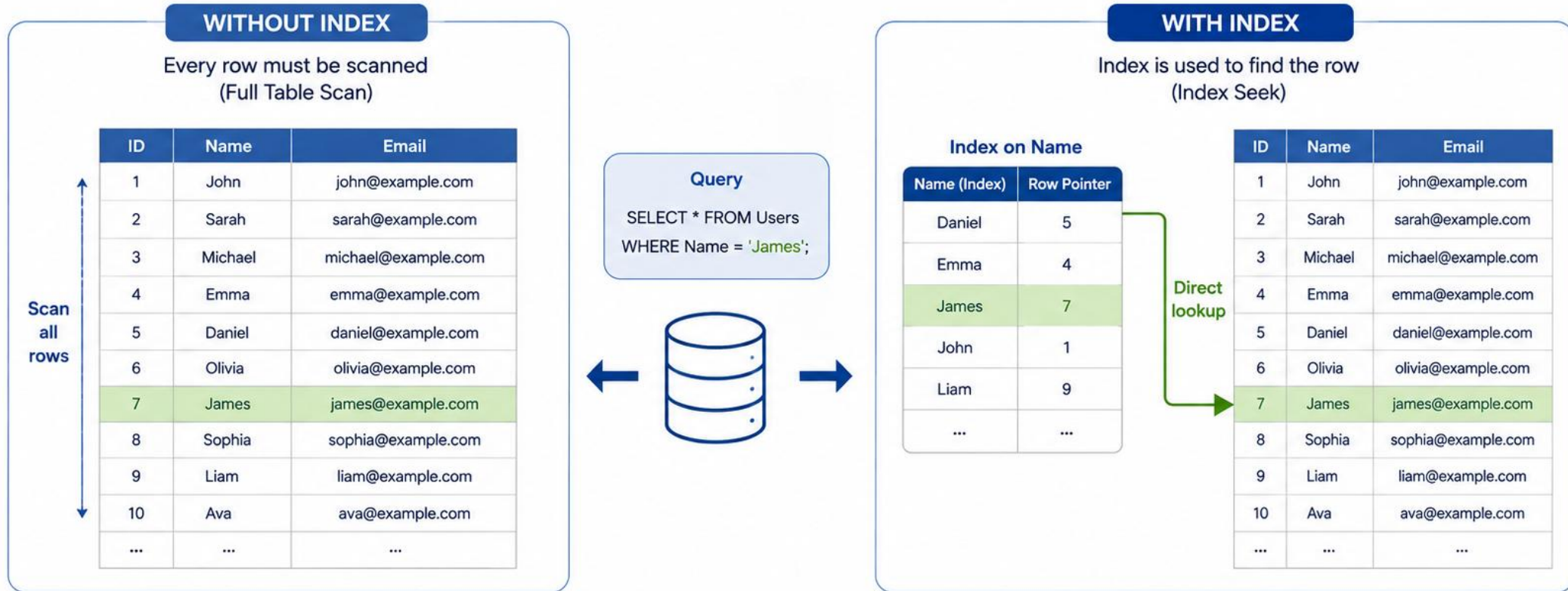
TRADE-OFFS & CONSIDERATIONS

- Indexes improve reads but slow down writes (INSERT, UPDATE, DELETE).
- Indexes consume additional storage space.
- Too many indexes can degrade performance and increase maintenance overhead.
- Create indexes based on query patterns, not on every column.

KEY TAKEAWAY Indexes make queries faster by allowing the database to find data quickly — essential for large tables. Use them wisely to balance read performance and write overhead.

Understanding Indexes

Indexes are data structures that improve the speed of data retrieval operations on a database table.



BENEFITS



Faster Queries

Retrieves data quickly without scanning all rows.



Improved Performance

Especially useful for large tables and complex queries.



Efficient Sorting and Filtering

Speeds up ORDER BY, GROUP BY and WHERE operations.



Better Scalability

Maintains performance as data grows.

TRY IT YOURSELF — EXERCISE 2: INDEX TRADEOFFS

Reinforces: the Trade-Offs & Considerations you just saw on Understanding Indexes, applied to the real CRM write path.

STEPS (DOCUMENT THE TRADEOFF)

Step	What To Do
1 — Name the Benefit	Write the read-side win: faster status filters and faster account-by-customer joins.
2 — Name the Cost	Write the write-side cost: slower INSERT/UPDATE for the Amina/Ravi seed rows at scale, plus more disk used per index.
3 — Plan Cleanup	Note that the lab may drop experimental indexes — plan to document a before/after for each one you keep or remove.
4 — State the Rule	Write the rule you will follow all lab: add an index only when EXPLAIN shows it is actually needed.

The rule to write down

```
-- Add an index only when EXPLAIN shows a Seq Scan that a real query pattern actually needs fixed.
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 2 before continuing: [exercises/exercise-02-index-tradeoffs.md](#).

CLUSTERED VS NON-CLUSTERED INDEXES

Both index types improve query performance, but they store and organize data differently.

KEY DIFFERENCES

Feature	Clustered Index	Non-Clustered Index
Data storage	Data rows stored at the leaf level.	Pointers (row locators) stored at the leaf level.
Physical order	Determines the physical order of data.	Does not affect physical order.
Number per table	Only one.	Multiple allowed.
Performance	Faster for range queries and sorting.	Faster for searches, filters, and joins.
Space overhead	Lower.	Higher.

WHEN TO USE A CLUSTERED INDEX

- Column is unique and frequently used in range queries.
- Table is frequently read in sorted order.
- Primary key columns (automatically clustered in many databases).

WHEN TO USE A NON-CLUSTERED INDEX

- Columns used in WHERE, JOIN, GROUP BY, ORDER BY.
- Columns with high selectivity (return fewer rows).
- When you need multiple indexes on one table.

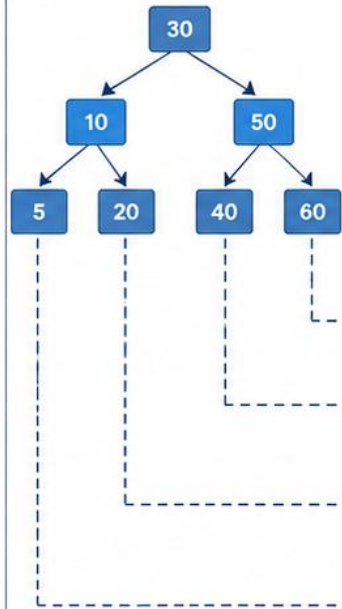
KEY TAKEAWAY Clustered Index = data is stored in index order (only one per table). Non-Clustered Index = index points to data (many allowed). Choose the clustered index column carefully — usually the primary key.

Clustered vs Non-Clustered Indexes

CLUSTERED INDEX

The data rows are stored in the same order as the clustered index key. Only one clustered index per table.

Index Structure (B-Tree)



Data Pages (Leaf Level)

Page 1	ID (Clustered Key)	Name	Email	Salary
	5	Amit	amit@example.com	60000
Page 2	ID (Clustered Key)	Name	Email	Salary
	10	Ben	ben@example.com	70000
Page 3	ID (Clustered Key)	Name	Email	Salary
	20	Chris	chris@example.com	80000
Page 4	ID (Clustered Key)	Name	Email	Salary
	30	David	david@example.com	90000
Page 5	ID (Clustered Key)	Name	Email	Salary
	40	Eve	eve@example.com	75000
Page 6	ID (Clustered Key)	Name	Email	Salary
	50	Frank	frank@example.com	65000
Page 7	ID (Clustered Key)	Name	Email	Salary
	60	Gaurav	gaurav@example.com	55000

Key Points

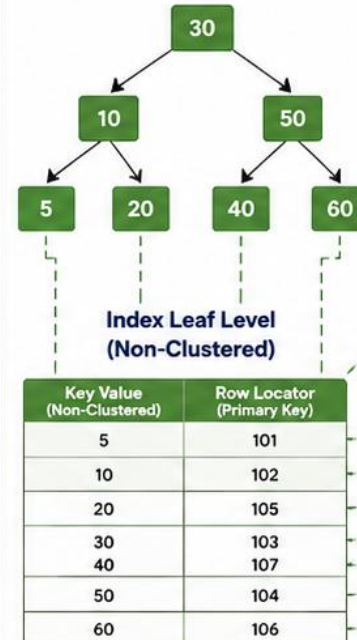
- Data is physically stored in the order of the clustered index key.
- Faster range queries and ORDER BY on the clustered key.
- Only one clustered index per table.



NON-CLUSTERED INDEX

The index is stored separately from the data. Pointers (row locators) point to the actual data rows.

Index Structure (B-Tree)



Data Pages (Heap / Unordered)

Page A	ID (Primary Key)	Name	Email	Salary
	101	Amit	amit@example.com	60000
Page B	ID (Primary Key)	Name	Email	Salary
	104	Frank	frank@example.com	65000
Page C	ID (Primary Key)	Name	Email	Salary
	107	Eve	eve@example.com	75000
Page D	ID (Primary Key)	Name	Email	Salary
	102	Ben	ben@example.com	70000
Page E	ID (Primary Key)	Name	Email	Salary
	105	Chris	chris@example.com	80000
Page F	ID (Primary Key)	Name	Email	Salary
	103	David	david@example.com	90000
Page G	ID (Primary Key)	Name	Email	Salary
	106	Gaurav	gaurav@example.com	55000

Key Points

- Index is separate from data; contains key + row locator.
- More indexes can be created on a table.
- Extra lookup needed to fetch actual data (slower for lookups).



TRY IT YOURSELF — EXERCISE 3: FILL SQL/INDEX TODOS (STARTER)

Reinforces: CREATE INDEX syntax plus sargable filtering and paging — a fill-in-the-blank starter, done on paper, not run against Postgres.

STARTER FILE — notes/lab38-todos.sql (FILL EVERY BLANK)

```
-- baseline (avoid)
SELECT * FROM customer WHERE ____ (full_name) = 'amina khan';

-- optimized lookup
SELECT customer_id, full_name, status FROM customer WHERE customer_id = ____;

-- supporting index ideas
CREATE INDEX ____ ON customer (status);
CREATE INDEX ____ ON account (customer_id);

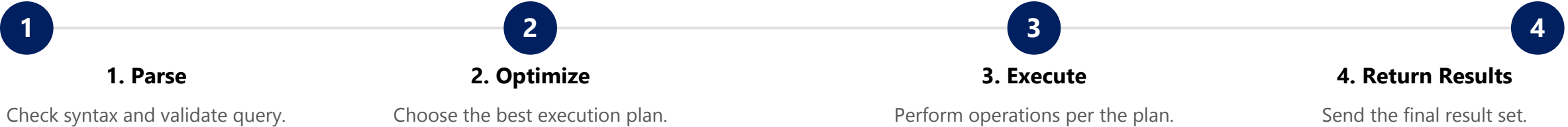
-- paging sketch
SELECT customer_id, full_name FROM customer ORDER BY customer_id LIMIT ____ OFFSET ____;
```

Blank	Suggested Fill
Baseline function / lookup value	lower — this is the non-sargable pattern to recognize, not repeat.
customer_id filter value	'CUS-1001' — Amina Khan's fixture.
Two index names	idx_customer_status, idx_account_customer.
LIMIT / OFFSET	20 / 0 — plus a // TODO comment: prefer keyset pagination (WHERE customer_id > :last) for deep pages.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 3 (STARTER) before continuing: exercises/exercise-03-fill-sql-index-todos.md. This is a starter file, not a finished solution — replace every ____ and // TODO, and do not run it against Postgres yet; Lab 38 will measure it for real.

QUERY EXECUTION PLANS (1 OF 2)

A query execution plan shows how the database engine will execute a SQL query to return results — the roadmap for every tuning decision.



WHY EXECUTION PLANS MATTER

- Understand how the database retrieves data.
- Identify performance bottlenecks.
- Verify index usage.
- Compare different query versions.
- Make informed tuning decisions.

COMMON OPERATORS IN EXECUTION PLANS

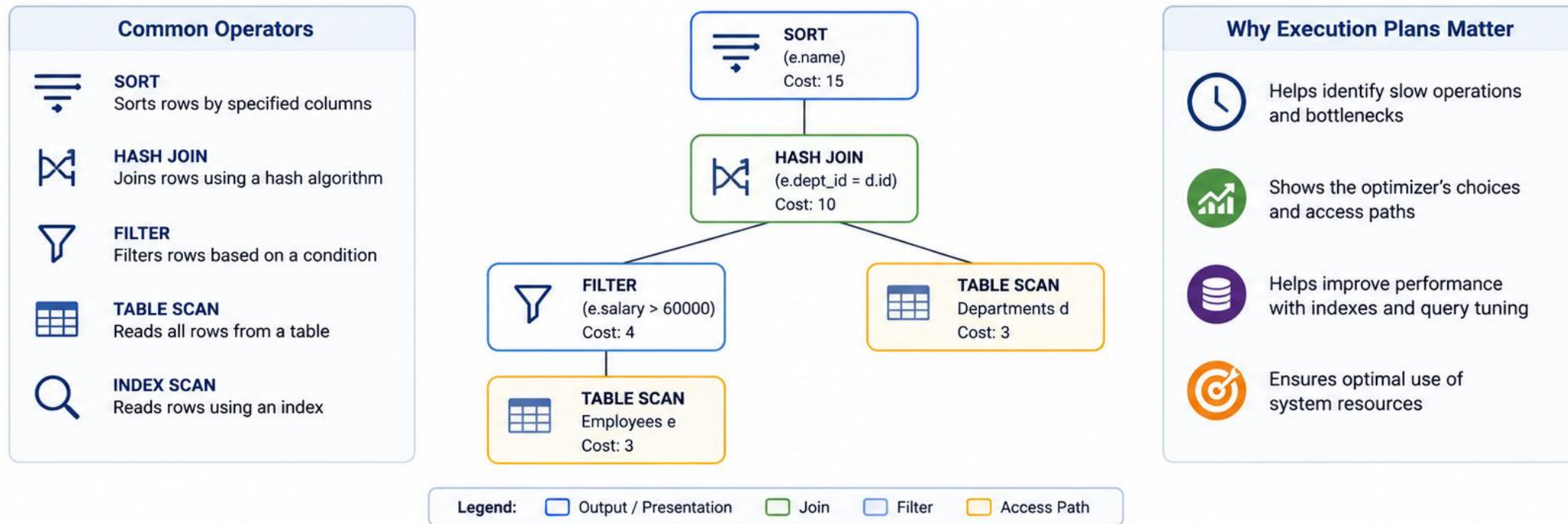
Operator	Meaning
Seq Scan	Scans the entire table row by row; used when no index helps.
Index Scan / Seek	Uses an index to find matching rows — faster than a seq scan.
Nested Loop Join	For each outer row, finds matching inner rows.
Hash Join	Builds a hash table on one input and probes with the other.
Sort	Sorts rows based on specified columns.
Aggregate	Performs functions like COUNT, SUM, AVG.

KEY TAKEAWAY A query execution plan reveals exactly how the database retrieves your data — parse, optimize, execute, return. Reading it is the first step to fixing a slow query.

Query Execution Plans



Example Execution Plan (Plan Tree)



QUERY EXECUTION PLANS (2 OF 2)

Reading a real EXPLAIN ANALYZE plan turns a black box into a concrete, actionable tuning target.

EXAMPLE QUERY

```
SELECT e.name, d.dept_name FROM employees e JOIN departments d
ON e.dept_id = d.dept_id WHERE e.salary > 50000 ORDER BY e.name;
```

SAMPLE EXECUTION PLAN — EXPLAIN ANALYZE (POSTGRESQL)

```
Sort (cost=25.50..25.53 rows=12 width=64)
  Sort Key: e.name
  -> Hash Join (cost=0.42..25.32 rows=12 width=64)
    Hash Cond: (e.dept_id = d.dept_id)
    -> Seq Scan on employees e (cost=0.00..18.20 rows=20)
      Filter: (salary > 50000)
    -> Hash (cost=0.26..0.26 rows=26 width=36)
      -> Seq Scan on departments d (cost=0.00..0.26 rows=26)
```

READING THE PLAN — KEY COLUMNS

Column	Description
cost	Estimated cost to execute the operation (startup..total).
rows / width	Estimated number of rows returned and estimated row size in bytes.
actual time / loops	(With ANALYZE) real time taken and how many times the operation ran.

KEY TAKEAWAY Seq Scan applies the salary filter, Hash Join combines employees and departments, Sort orders by name — index WHERE/JOIN/ORDER BY columns and keep statistics current with ANALYZE.

TRY IT YOURSELF — EXERCISE 4: EXPLAIN CHECKLIST

Reinforces: reading the EXPLAIN ANALYZE plan you just saw, built into a reusable PostgreSQL-native checklist for Lab 38.

STEPS (BUILD THE CHECKLIST)

Step	What To Do
1 — Write the Command	Write the exact command you'll run in Lab 38: EXPLAIN (ANALYZE, BUFFERS) <sql>;
2 — Name What to Look For	List Seq Scan vs. Index Scan, the estimated rows, and the buffers line in your checklist.
3 — Name a Success Signal	Write: an Index Scan on customer_id for the Amina lookup is a good sign.
4 — Note the PostgreSQL Command	Write that ANALYZE customer; refreshes statistics in PostgreSQL — this is not DBMS_STATS.

The exact command

```
EXPLAIN (ANALYZE, BUFFERS) SELECT customer_id, full_name, status FROM customer WHERE customer_id = 'CUS-1001';
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 4 before continuing: [exercises/exercise-04-explain-checklist.md](#) (workspace: [examples/module-38-exercises](#)).

IDENTIFYING SLOW QUERIES (1 OF 2)

Slow queries degrade application performance and user experience. Identifying them early helps you optimize databases and ensure responsive systems.

WHAT IS A SLOW QUERY?

- Takes longer than expected to execute and consumes more database resources (CPU, I/O, memory).
- Impact: increased response time, higher load, timeouts, and poor user experience.

COMMON CAUSES

- Missing or ineffective indexes.
- Full table scans.
- Complex joins and subqueries.
- Unoptimized WHERE conditions.
- Large result sets.
- Outdated statistics.

WHERE TO LOOK

- **Database Logs** — enable the slow query log (e.g., `log_min_duration_statement` in PostgreSQL).
- **Query Monitoring Tools** — pgAdmin, DataGrip, DBeaver, or APM tools.
- **Application Logs** — identify queries taking too long in app logs.
- **Database Views** — system views for long-running or expensive queries.

PROCESS TO IDENTIFY SLOW QUERIES

- 1 **Detect**
- 2 **Capture**
- 3 **Analyze**
- 4 **Prioritize**
- 5 **Verify**

KEY TAKEAWAY A slow query takes longer than expected and consumes excess CPU, I/O, and memory — identify it early through logs, monitoring tools, and a repeatable detect-capture-analyze-verify process.

Identifying Slow Queries



IDENTIFYING SLOW QUERIES (2 OF 2)

PostgreSQL ships built-in views and commands to find and analyze slow queries; combine them with disciplined best practices.

USEFUL POSTGRESQL VIEWS & COMMANDS

View / Command	What It Does
pg_stat_statements	Find top queries by total time, mean time, calls.
EXPLAIN / EXPLAIN ANALYZE	Show a query's execution plan and cost.
pg_stat_activity	View currently running queries and their duration.
log_min_duration_statement	Log queries that exceed a specified time (ms).
auto_explain extension	Automatically log slow queries with execution plans.

Example: Finding Slow Queries

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
-- Top 5 slow queries by total time
SELECT query, calls, total_time, mean_time, rows
FROM pg_stat_statements
ORDER BY total_time DESC LIMIT 5;
```

WHAT TO CHECK IN THE EXECUTION PLAN

- Seq Scan instead of an Index Scan.
- High-cost operations (Sort, Hash, Materialize).
- Large gap between estimated vs. actual row count.
- Nested Loop on large datasets.
- Expensive filters executed after joins.

BEST PRACTICES

- Always use indexes on filter, join, and sort columns.
- Avoid SELECT *; fetch only required columns.
- Keep statistics updated (ANALYZE regularly).
- Use LIMIT for large result sets when applicable.
- Monitor and review slow queries continuously.

KEY TAKEAWAY Identify slow queries early with pg_stat_statements and EXPLAIN ANALYZE, analyze their root cause, and optimize for better performance.

QUERY TUNING STRATEGIES (1 OF 2)

Query tuning improves the performance of SQL queries to reduce response time and resource usage. Good queries are correct, efficient, scalable, and maintainable.

Faster Response Time

Lower Resource Usage

Higher Throughput

Better Scalability

Cost Efficiency



1. Use Indexes Effectively

Create indexes on WHERE/JOIN/ORDER BY/GROUP BY columns. Avoid over-indexing; use composite indexes for multi-column filters.



2. Write Efficient Queries

Select only required columns. Avoid functions on indexed columns. Prefer EXISTS over IN for large subqueries. Use LIMIT/OFFSET carefully.



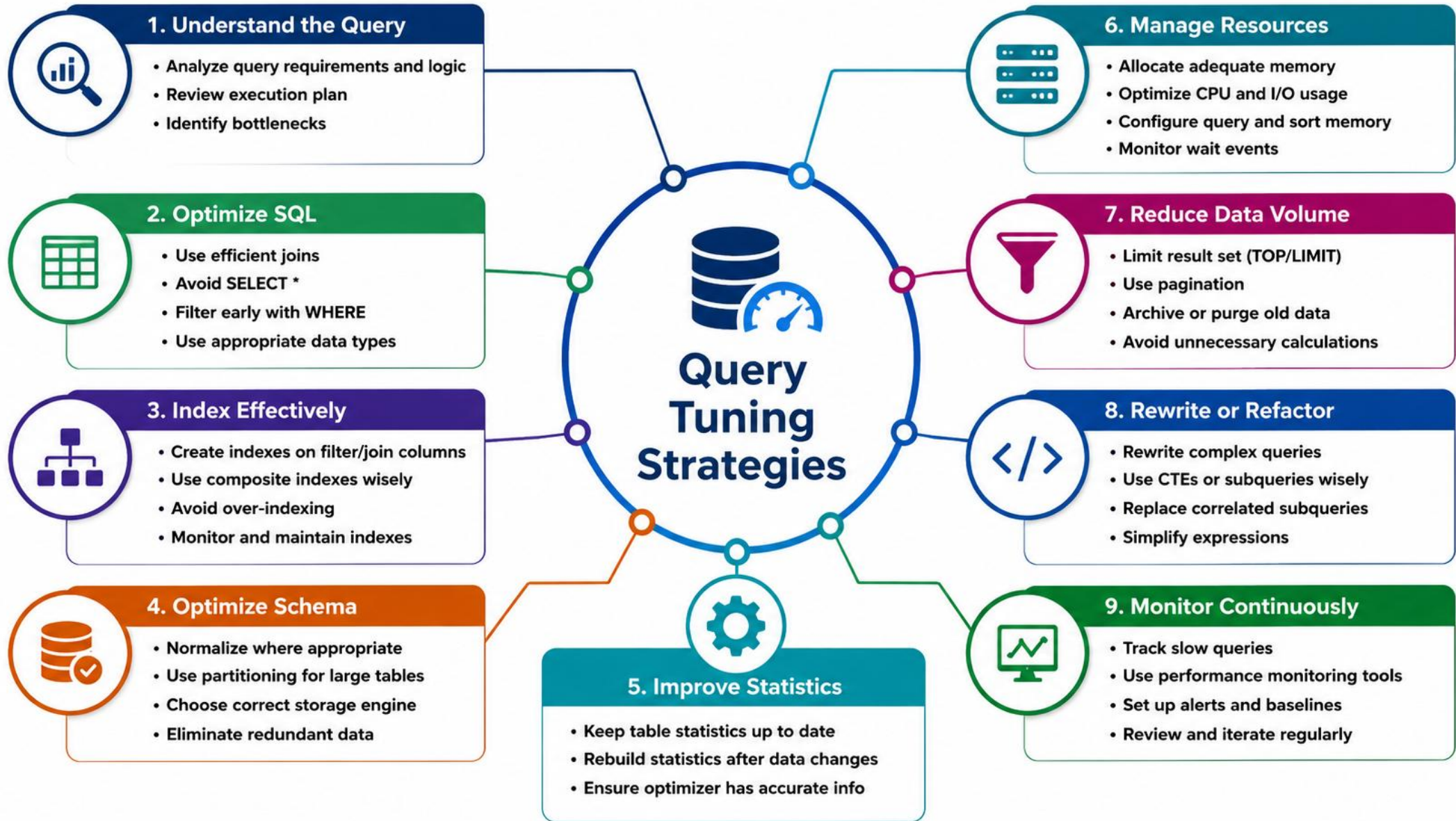
3. Optimize Joins

Ensure join columns are indexed on both tables. Join smaller tables first. Prefer INNER JOIN when possible; avoid Cartesian products.

EXAMPLE — INEFFICIENT VS. EFFICIENT JOIN

```
-- Inefficient: SELECT * FROM orders o, customers c WHERE o.customer_id = c.id;
-- Efficient:   SELECT o.order_id, c.name FROM orders o
--              INNER JOIN customers c ON o.customer_id = c.id;
```

KEY TAKEAWAY Good query performance is critical for scalable, reliable applications — start by indexing effectively, writing efficient queries, and optimizing joins.



QUERY TUNING STRATEGIES (2 OF 2)

Four more strategies — reducing data processed, fresh statistics, execution plan analysis, and general best practices — complete the tuning toolkit.



4. Reduce Data Processed

Filter early with strong WHERE clauses. Partition large tables. Archive old data. Avoid unnecessary DISTINCT / ORDER BY on large result sets.



5. Update Statistics Regularly

The optimizer relies on statistics to choose the best plan. Outdated statistics lead to poor plans — run ANALYZE (PostgreSQL) regularly.



6. Analyze Execution Plans

Use EXPLAIN / EXPLAIN ANALYZE. Look for full scans, missing indexes, and expensive sorts/ hashes. Fix root causes, not symptoms.



7. Other Best Practices

Use prepared statements and bind variables. Avoid functions on indexed columns. Normalize data. Monitor and tune continuously.

EXAMPLE TUNING WORKFLOW

1

1. Identify Slow Query

2

2. Analyze Plan

3

3. Apply Strategies

4

4. Test & Validate

5

5. Monitor Continuously

KEY TAKEAWAY Use a combination of indexes, efficient SQL, and regular maintenance — always measure, analyze, and validate improvements. Small changes can lead to big performance gains.

AVOIDING FULL TABLE SCANS (1 OF 2)

A full table scan (sequential scan) reads every row in a table to find the required data. It is expensive and does not scale.

WHAT IS A FULL TABLE SCAN?

- The database reads every row sequentially.
- Used when no usable index exists.
- Happens with missing/unused indexes, functions on columns, or non-sargable predicates.
- Expensive and slow for large tables.

WHY AVOID FULL TABLE SCANS?

Slower Performance

High Resource Usage

Poor Scalability

Higher Cost

Bad User Experience

GOAL

Design queries and indexes so the database finds only the rows it needs — never the whole table.

COMMON REASONS FOR FULL TABLE SCANS

Reason	Explanation
Missing Index	No index on columns used in WHERE, JOIN, ORDER BY, or GROUP BY.
Functions on Columns	Wrapping a column in a function (e.g., UPPER(name)) prevents index usage.
Leading Wildcard	LIKE '%abc' cannot use a standard index.
Data Type Mismatch	Comparing different data types causes index invalidation.
Outdated Statistics	Optimizer may choose a full scan if statistics are stale.

KEY TAKEAWAY A full table scan reads every row and does not scale — it happens when no usable index exists, functions wrap indexed columns, or statistics are stale.

Avoiding Full Table Scans

❌ FULL TABLE SCAN (AVOID)

What Happens?

- Database reads every row in the table
- High I/O and CPU usage
- Slower query performance
- Does not scale with large data

```
SELECT * FROM Orders  
WHERE status = 'Shipped';
```

Scan All Rows

OrderID	CustomerID	Status	Amount
1	101	Pending	250
2	102	Shipped	125
3	103	Cancelled	300
...	...	...	...
1000000	99999	Pending	150

Reads every row

 More Time
Higher Cost

✅ INDEX SCAN (PREFERRED)

What Happens?

- Uses an index to find matching rows
- Reads only relevant rows
- Lower I/O and CPU usage
- Faster and scalable performance

```
SELECT * FROM Orders  
WHERE status = 'Shipped';
```

Use Index on Status

Index: idx_orders_status	
Status	Row Pointers
Cancelled	→ 3, 8, 15, ...
Pending	→ 1, 5, 7, ...
Shipped	→ 2, 10, 25, ...

OrderID	CustomerID	Status	Amount
2	102	Shipped	125
10	105	Shipped	220

 Less Time
Lower Cost

Best Practices to Avoid Full Table Scans



Use Indexes

Create indexes on columns used in WHERE, JOIN, ORDER BY, and GROUP BY.



Filter Early

Use WHERE clauses to filter data as early as possible.



Select Only What You Need

Avoid SELECT *.
Select only required columns.



Keep Statistics Updated

Update table and index statistics for better query plans.



Analyze Execution Plans

Use EXPLAIN or execution plans to identify full scans.



Design for Performance

Proper schema design reduces the need for full scans.

AVOIDING FULL TABLE SCANS (2 OF 2)

Sargable queries — ones the optimizer can match directly to an index — are the key to avoiding full table scans.

HOW TO AVOID FULL TABLE SCANS

- Create indexes on columns used in filtering, joining, sorting, and grouping.
- Write sargable queries (search ARGument-ABLE).
- Avoid functions on indexed columns in the WHERE clause.
- Use leading values in LIKE ('abc%' instead of '%abc').
- Use appropriate, consistent data types and comparisons.
- Keep statistics up to date (ANALYZE).

EXAMPLE — MAKING A QUERY SARGABLE

```
-- Non-sargable (causes full scan)
SELECT * FROM employees WHERE UPPER(name) = 'AMIT';
-- Sargable (uses index)
SELECT * FROM employees WHERE name = 'Amit';
```

HOW TO DETECT FULL TABLE SCANS

- **Execution Plan** — look for 'Seq Scan' in the plan.
- **Slow Query Logs** — identify queries with high duration and rows examined.
- **Monitoring Tools** — pg_stat_statements, EXPLAIN ANALYZE, APM tools.
- **High Rows Examined** — compare rows examined vs. rows returned.

BEST PRACTICES

- Design indexes based on real query patterns.
- Review slow queries and execution plans regularly.
- Avoid SELECT *; select only required columns.
- Partition large tables when appropriate.
- Monitor and tune queries continuously.

KEY TAKEAWAY Full table scans are one of the biggest performance killers. Sargable queries, proper indexing, fresh statistics, and regular monitoring keep the database reading only what it needs.

TRY IT YOURSELF — EXERCISE 5: SARGABILITY

Reinforces: classifying real predicates as sargable or not for PostgreSQL, using the UPPER(name) pattern you just saw.

REFERENCE TABLE — CLASSIFY EACH PREDICATE

Predicate	Sargable?
customer_id = 'CUS-1001'	Yes
status = 'ACTIVE'	Yes (with index)
LOWER(full_name) = 'amina khan'	Usually no on a plain index
created_at >= TIMESTAMP '2026-01-01'	Yes (range)
date_trunc('day', created_at) = ...	Often weaker than a range

STEPS

- **1 — Copy the table** — reproduce the reference table above into your notes.
- **2 — Rewrite one** — rewrite a non-sargable name search into something index-friendlier (store lowercased column, or use ILIKE carefully).
- **3 — State the range rule** — prefer created_at >= d AND created_at < d+1 over wrapping the column in a function.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 5 before continuing: [exercises/exercise-05-sargability.md](#). If old materials say TRUNC(created_at), map it to this PostgreSQL range/date_trunc contrast.

EFFICIENT QUERY DESIGN (1 OF 2)

Efficient query design returns correct results using the least amount of time, resources, and I/O. Well-designed queries are readable, maintainable, and optimized.

Faster Performance

Lower Resource Usage

Better Scalability

Improved Reliability

Cost Efficiency



1. Select Only What You Need

Avoid SELECT *. Select only required columns to reduce data transfer and processing.



2. Use Proper Filtering

Use WHERE to filter early. Avoid functions on indexed columns.



3. Optimize Joins

Join only necessary tables. Ensure join columns are indexed. Use INNER JOIN unless an outer join is required.



4. Avoid Unnecessary Sorting

Use ORDER BY only when needed. Ensure sorting columns are indexed. Avoid sorting large result sets.

EXAMPLE — RETURN LESS DATA, FASTER PERFORMANCE

```
-- Inefficient: SELECT * FROM employees;  
-- Efficient:   SELECT emp_id, name, salary FROM employees;
```

KEY TAKEAWAY Efficient query design is about doing less work and returning results faster — select only what you need, filter early, optimize joins, and avoid unnecessary sorting.

Efficient Query Design

Write queries that return the right data,
the right way, for the right performance.



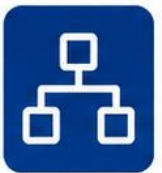
1. Select Only What You Need

Avoid SELECT *.
Retrieve only the columns
required for the task.



2. Filter Early

Use WHERE clauses to filter
rows as early as possible and
reduce the result set.



3. Use Proper Indexes

Index columns used in WHERE,
JOIN, ORDER BY, and GROUP BY
to speed up data retrieval.



4. Write Optimal Joins

Join tables on indexed columns.
Use appropriate JOIN types and
avoid unnecessary joins.



5. Avoid Unnecessary Sorting and Grouping

Sort and group only when needed.
Ensure required indexes support
ORDER BY and GROUP BY.



6. Review and Optimize

Analyze execution plans,
monitor performance, and
refactor queries regularly.



Better Performance

Faster response times and
efficient resource usage.



Lower Resource Usage

Reduces CPU, memory,
and I/O consumption.



Scalability

Well-designed queries scale
with growing data volumes.



Accurate Results

Returns the right data
without unnecessary overhead.

EFFICIENT QUERY DESIGN (2 OF 2)

Four more techniques — smart aggregation, appropriate data types, avoiding subquery pitfalls, and leveraging indexes — complete a disciplined design workflow.



5. Use Aggregates Wisely

Filter rows before aggregation. Aggregate only necessary data. Avoid DISTINCT if GROUP BY is enough.



6. Use Appropriate Data Types

Use the smallest data type that fits. Avoid large types on indexed columns. Consistent types prevent implicit conversions.



7. Avoid Subquery Pitfalls

Use JOINS instead of correlated subqueries where possible. Ensure subqueries return only necessary rows.



8. Leverage Indexes

Create indexes on WHERE/JOIN/ORDER BY/GROUP BY columns. Avoid functions on indexed columns. Don't over-index.

EFFICIENT QUERY DESIGN WORKFLOW

1

1. Requirements

2

2. Filter Early

3

3. Join Smartly

4

4. Select Wisely

5

5. Check & Tune

6

6. Test & Monitor

KEY TAKEAWAY Efficient query design is not just about writing correct SQL — it's about writing smart SQL that does less work and returns results faster. Follow the workflow: filter, join, select, then test and tune.

DATABASE PERFORMANCE BEST PRACTICES (1 OF 2)

High-performing databases result from good design, efficient queries, proper indexing, regular maintenance, and continuous monitoring.

Better Performance

Efficient Resource Usage

Scalability

Reliability & Availability

Cost Efficiency



1. Design for Performance

Use appropriate data types, normalize data, choose the right keys, and consider partitioning for large tables.



2. Write Efficient Queries

Select only required columns, filter early with WHERE, use EXISTS over IN, use LIMIT/OFFSET carefully.



3. Use Indexes Wisely

Create indexes on WHERE/JOIN/ORDER BY columns; avoid over-indexing; rebuild or reorganize indexes regularly.



4. Avoid Expensive Operations

Avoid functions on indexed columns, implicit conversions, unnecessary sorting/DISTINCT, and Cartesian products.



5. Keep Statistics Up to Date

Update table and index statistics regularly — outdated statistics lead to poor execution plans.

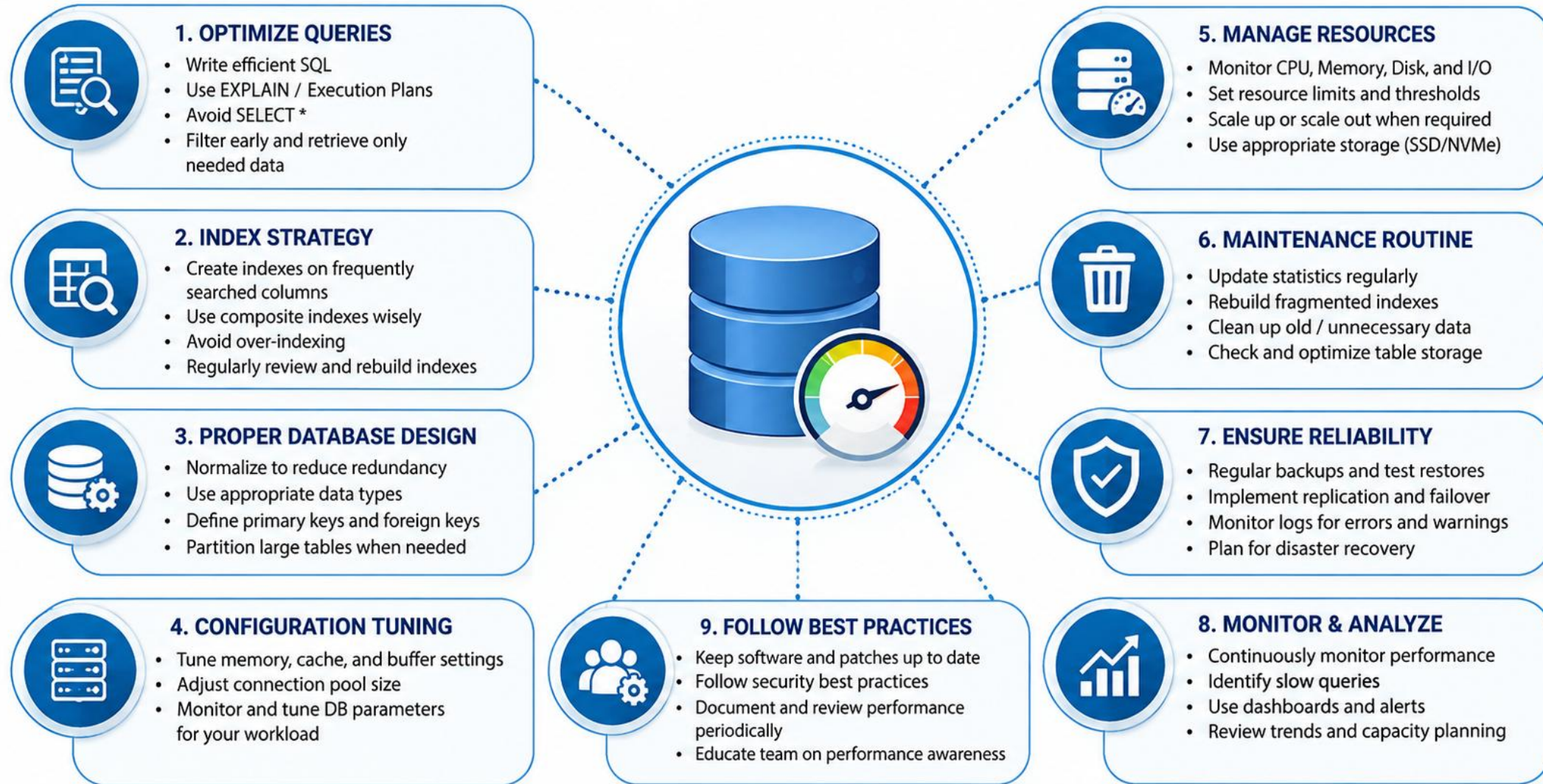


6. Manage Resources

Monitor CPU/memory/disk/I/O usage, configure database parameters appropriately, use connection pooling.

KEY TAKEAWAY High-performing databases start with good design, efficient queries, proper indexing, and up-to-date statistics — small, consistent practices compound into big performance wins.

Database Performance Best Practices



DATABASE PERFORMANCE BEST PRACTICES (2 OF 2)

The remaining six best practices — monitoring, transactions, maintenance, security, testing, and capacity planning — round out a production-grade performance program.



7. Monitor Continuously

Monitor slow queries and wait events; track performance metrics; set up alerts for anomalies; use dashboards.



8. Optimize Transactions

Keep transactions short; commit as early as possible; avoid long-running transactions.



9. Implement Proper Maintenance

Rebuild/reorganize indexes; clean up unused objects; archive old data; check and fix fragmentation.



10. Secure Your Database

Use strong passwords and roles; follow least privilege; encrypt data at rest and in transit; audit access.



11. Test and Tune Regularly

Test queries with realistic data volumes; review execution plans periodically; validate changes in lower environments.



12. Plan for Growth

Monitor growth trends; plan capacity for storage/memory/connections; use partitioning and archiving.

PERFORMANCE CHECKLIST

Slow queries reviewed

Indexes optimized

Statistics updated

Resources monitored

Maintenance performed

Security enforced

Growth planned

KEY TAKEAWAY Good database performance results from smart design, efficient queries, proper indexing, regular maintenance, and continuous monitoring — applied consistently to deliver fast, reliable, scalable solutions.

PRODUCTION SQL MONITORING (1 OF 2)

Production SQL monitoring helps you continuously track database health, detect issues early, and ensure consistent performance for your applications.

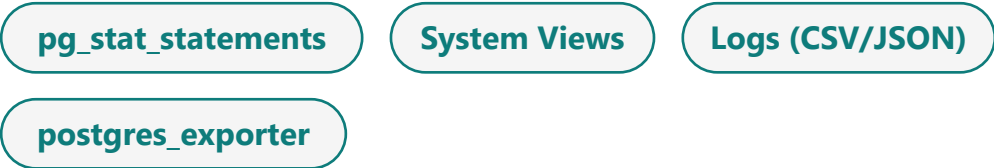
WHY MONITOR IN PRODUCTION?

- **Ensure Performance** — identify slow queries and bottlenecks before users are impacted.
- **Improve Availability** — detect locks, blocking, and resource contention early.
- **Reduce Costs** — optimize resource usage and avoid over-provisioning.
- **Capacity Planning** — understand trends to plan for growth and workload changes.

PRODUCTION MONITORING ARCHITECTURE



COLLECTION METHODS

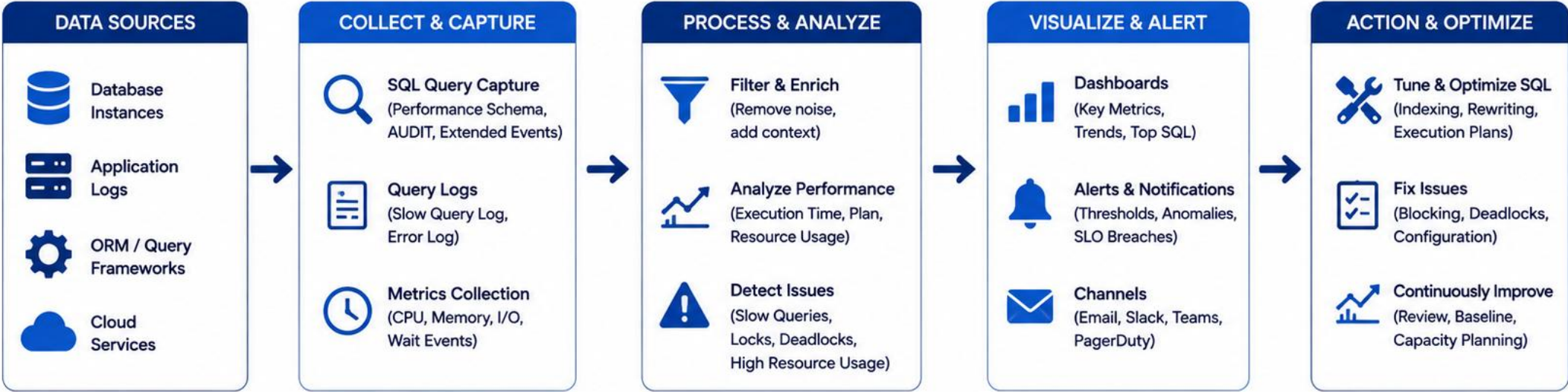


KEY METRICS TO MONITOR

Metric	What It Tells You
Query Response Time (avg, p95, p99)	Overall query performance experienced by users.
CPU Utilization / Memory Usage	CPU pressure and buffer-cache / memory pressure on the database.
Lock Waits / Blocking / Deadlocks	Contention between concurrent queries and transactions.

KEY TAKEAWAY Monitor query response time, CPU/memory, and lock waits — the goal is to detect issues before users are impacted, not after.

Production SQL Monitoring



KEY METRICS TO MONITOR						
Query Response Time (Avg / 95th / 99th Percentile)	Throughput (Queries per Second)	Top SQL (By Time, CPU, Reads)	Lock & Waits (Blocking Sessions, Wait Events)	Resource Usage (CPU, Memory, I/O, Temp Space)	Errors (Errors, Timeouts, Aborts)	Trends & Baselines (Compare over Time)

COMMON TOOLS							
Prometheus	Grafana	Datadog	New Relic	Percona Monitoring	pg_stat_statements (PostgreSQL)	ORACLE OEM / AWR	SQL Server DMVs

PRODUCTION SQL MONITORING (2 OF 2)

Essential tools, alerting thresholds, and dashboards turn raw metrics into an early-warning system for database health.

ESSENTIAL TOOLS & SOURCES

- **pg_stat_statements** — tracks execution stats for all normalized queries.
- **System Views** — pg_stat_activity, pg_locks, pg_stat_io, pg_stat_database.
- **Logs** — log_min_duration_statement captures slow queries.
- **Monitoring Tools** — pgAdmin, DataGrip, DBeaver, Prometheus + Grafana, Datadog.

BEST PRACTICES

- Enable pg_stat_statements in all environments.
- Set and review slow query thresholds regularly.
- Correlate database metrics with application performance.
- Automate alerting and incident response.

COMMON ALERTS TO CONFIGURE

```
-- Slow queries: mean_time > 1000 ms
-- High CPU: cpu_usage > 80% for 5m
-- Lock waits: wait_event_type = 'Lock'
-- Deadlocks: deadlocks_total increases
-- Replication lag above threshold
```

DASHBOARDS TO INCLUDE

Top 10 Slowest Queries

Database Load

Active Connections

Lock Waits

Replication Status

Errors & Deadlocks

KEY TAKEAWAY Monitoring in production is not optional — it's essential for reliability, performance, and user satisfaction. Measure. Alert. Analyze. Optimize.

TRY IT YOURSELF — EXERCISE 6: LAB 38 READINESS (CAPSTONE)

Reinforces: every concept from this module, pulled into one final self-check before you open the graded Lab 38 guide.

STEPS (PRE-LAB SELF-CHECK)

Step	What To Do
1 — Write a Hypothesis	Write: customer_id equality uses an index; LOWER(full_name) may seq scan.
2 — Preview the Evidence	Note where it will live: future EXPLAIN snippets get pasted into database/performance/report.md.
3 — Confirm the Dialect	Write that your primary tools are PostgreSQL EXPLAIN / ANALYZE — not Oracle packages like DBMS_XPLAN.
4 — Mark Your Pass	Pass this exercise once every SQL TODO from Exercise 3 is filled in.

The hypothesis to write down

```
customer_id equality uses an index; LOWER(full_name) may seq scan.
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 6 before continuing: exercises/exercise-06-lab38-readiness.md. This is the last pre-lab check — once it passes, open lab38/LAB-38-GUIDE.md.

LAB OVERVIEW — SQL AND QUERY PERFORMANCE

Lab 38: PostgreSQL SQL and CRM Query Performance — DBMS_XPLAN, Indexes, Sargable Predicates

BUSINESS SCENARIO

- Before Spring Data JPA wires the CRM API to PostgreSQL (Lab 39), leadership freezes a performance gate: no 'add index because it felt slow' without actual plans, before/after numbers, and a report naming which query was measured.
- You own that gate for list-by-status, email lookup, customer→account joins, and deep paging for Amina (CUS-1001 ACTIVE), Ravi (CUS-1002 PROSPECT), and bulk synthetic rows.
- Guesswork indexes waste storage and slow writes — this lab makes you earn every retained index with evidence.

LEARNING OBJECTIVES

- Generate $\geq 50,000$ CRM rows and gather optimizer statistics.
- Capture actual plans with `DBMS_XPLAN.DISPLAY_CURSOR (ALLSTATS LAST)`.
- Create indexes for measured queries — not speculative ones.
- Rewrite non-sargable `TRUNC()` predicates into half-open date ranges.
- Compare nested-loop vs. hash-join strategies by selectivity.
- Implement deterministic `OFFSET/FETCH` and efficient keyset paging.
- Challenge every index: drop, re-plan, keep only justified ones.

WHAT YOU BUILD

- lab38-crm, copied from Lab 37: $\geq 50k$ customers with $\sim 70/30$ ACTIVE/PROSPECT skew; `DBMS_STATS` baseline; `ux_customer_email_norm` unique index; `ix_customer_status_created` composite index; a sargable date-range rewrite; `ix_account_customer` join index; deterministic `OFFSET + keyset` paging; an index challenge cycle; and `database/performance/report.md`.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor every measured query. Every retained index needs a before/after plan, buffer count, and median time.

LAB TASKS AND DELIVERABLES

PostgreSQL SQL and CRM Query Performance — implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Branch Lab 37 → scaffold lab38-crm scripts 01–05 + report.md headings.
2	Generate ≥50k customers, ~70/30 ACTIVE/PROSPECT skew; preserve CUS-1001/CUS-1002.
3	Gather optimizer statistics; record NUM_ROWS / LAST_ANALYZED and query binds.
4	Capture the actual baseline email-lookup plan (DBMS_XPLAN, ALLSTATS LAST).
5	Create ux_customer_email_norm unique index; re-measure the identical query.
6	Create ix_customer_status_created composite index; measure the ACTIVE list query.
7	Rewrite TRUNC(created_at) into a sargable half-open range; compare plans.
8	Compare nested-loop vs. hash-join on customer→account; add ix_account_customer.
9	Implement OFFSET + keyset paging; challenge each index and write report.md.

Sargable date rewrite (Step 7)

```
-- Non-sargable: WHERE TRUNC(created_at) = DATE '2026-07-01';
-- Sargable:      WHERE created_at >= TIMESTAMP '2026-07-01 00:00:00'
--               AND created_at <  TIMESTAMP '2026-07-02 00:00:00';
```

EXPECTED DELIVERABLES

- database/performance/01–05 SQL scripts.
- ≥50k load with documented skew + preserved CRM fixtures.
- Baseline and after-index DBMS_XPLAN evidence.
- Sargable date rewrite comparison.
- Join strategy notes (selective vs. broad).
- Deterministic OFFSET + keyset paging demos.
- Index challenge cycle with retained-index justification.
- report.md with plan hash, buffers, median time, write cost.

RUBRIC (100 MARKS)

- Environment/Structure 10 · Core Implementation 30 · Integration/Config 15 · Failure Handling 15 · Automated Verification 10 · Security/Prod Awareness 10 · Documentation 10

KEY TAKEAWAY Adding indexes without before/after plans loses evidence marks; keeping every experimental index without a challenge loses production-awareness marks; forcing a 'pretty' plan without explaining why loses honesty marks.

MODULE 38 SUMMARY

In this module, we explored how to analyze, tune, and monitor SQL queries to ensure optimal database performance and a great user experience.

KEY CONCEPTS COVERED



SQL & Joins

SELECT, WHERE, ORDER BY, and INNER/LEFT/RIGHT/FULL OUTER JOIN.



Indexes & Plans

Clustered/non-clustered indexes and reading EXPLAIN ANALYZE plans.



Identifying Slow Queries

pg_stat_statements, detect-capture-analyze-verify process.



Tuning & Sargability

Query tuning strategies and avoiding non-sargable full table scans.



Efficient Design

Filter, join, select, and aggregate with the least resource usage.



Best Practices & Monitoring

Twelve production practices and continuous SQL monitoring.

THE PERFORMANCE CYCLE

1

1. Analyze

2

2. Tune

3

3. Validate

4

4. Monitor

5

5. Improve

KEY TAKEAWAY High-performing databases are built through smart design, efficient queries, proper indexing, and proactive monitoring. Apply these concepts consistently to deliver fast, reliable, scalable solutions.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of execution plans, performance metrics, and the most common causes and fixes for slow queries.

1. Which command is used in PostgreSQL to view the execution plan of a query?

- A. SHOW PLAN
- B. EXPLAIN**
- C. ANALYZE QUERY
- D. TRACE PLAN

3. What is a common cause of slow queries?

- A. Using too many indexes
- B. Missing or improper indexes**
- C. Small result sets
- D. Having constraints

2. Which metric indicates the total time a query took to complete?

- A. Total Cost
- B. Execution Time**
- C. Cache Hit Ratio
- D. Rows Scanned

4. Which of the following helps avoid full table scans?

- A. Using SELECT *
- B. Creating appropriate indexes**
- C. Storing data in text format
- D. Disabling statistics

KEY TAKEAWAY EXPLAIN shows the execution plan; execution time measures total query duration; missing or improper indexes are a leading cause of slow queries; appropriate indexes help avoid full table scans.

KNOWLEDGE CHECK (2 OF 2)

*Four more questions on SELECT *, join efficiency, why statistics matter, and the benefit of monitoring pg_stat_statements.*

5. Why is SELECT * considered a bad practice?

- A. It improves performance
- B. It retrieves unnecessary columns**
- C. It creates table locks
- D. It prevents index usage

7. Statistics on tables and indexes are important because they:

- A. Reduce storage size
- B. Help the optimizer choose better plans**
- C. Increase CPU speed
- D. Encrypt the data

6. Which join type is generally the most efficient when a valid relationship is required on both sides?

- A. CROSS JOIN
- B. LEFT OUTER JOIN
- C. INNER JOIN**
- D. FULL OUTER JOIN

8. What is the main benefit of monitoring pg_stat_statements?

- A. It modifies query results
- B. It provides insights into query performance**
- C. It creates indexes automatically
- D. It encrypts queries

KEY TAKEAWAY SELECT * retrieves unnecessary columns; INNER JOIN is typically the most efficient join for a required relationship; statistics help the optimizer choose better plans; pg_stat_statements provides insight into query performance.

MODULE 38 COMPLETE

SQL and Query Performance

You can now write efficient SQL, master all four JOIN types, use indexes wisely, read execution plans, and apply tuning strategies to keep PostgreSQL databases fast, reliable, and scalable.